AI WEATHERWISE

12-Page Runnable Project & Source Code

Intelligent Weather Forecasting and Analysis System Using Weather API

Beginner-friendly Flask + Open-Meteo implementation with complete source code and simple run instructions.

Technology: Python • Flask • Requests • HTML5 • CSS3 • JavaScript • Open-Meteo

Note: Educational weather dashboard; not an official emergency warning system.

2. Folder Structure

```
AI_WeatherWise/
■■■ app.py
■■■ requirements.txt
■■■ run_windows.bat
■■■ run_linux_mac.sh
■■■ templates/
■   ■■■ index.html
■■■ static/
    ■■■ style.css
```

app.py: Flask server/API integration. **index.html:** interface/JavaScript. **style.css:** responsive styling. **requirements.txt:** packages. Run scripts are optional helpers.

3. Backend Source Code – app.py

```
from flask import Flask, render_template, request, jsonify
import requests

app = Flask(__name__)

GEOCODING_URL = "https://geocoding-api.open-meteo.com/v1/search"
WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/weather")
def weather():
    city = request.args.get("city", "").strip()

    if not city:
        return jsonify({"error": "Please enter a city name."}), 400

    try:
        geo = requests.get(
            GEOCODING_URL,
            params={"name": city, "count": 1, "language": "en", "format": "json"},
            timeout=10
        )
        geo.raise_for_status()
        geo_data = geo.json()

        if not geo_data.get("results"):
            return jsonify({"error": "Location not found."}), 404

        place = geo_data["results"][0]
        lat = place["latitude"]
        lon = place["longitude"]

        weather = requests.get(
            WEATHER_URL,
            params={
                "latitude": lat,
                "longitude": lon,
                "current": ",".join([
                    "temperature_2m",
                    "relative_humidity_2m",
                    "apparent_temperature",
                    "precipitation",
                    "weather_code",
                    "wind_speed_10m"
                ]),
                "daily": ",".join([
                    "weather_code",
                    "temperature_2m_max",
                    "temperature_2m_min",
                    "precipitation_probability_max"
                ]),
                "forecast_days": 5,
                "timezone": "auto"
            },
            timeout=10
        )
        weather.raise_for_status()
        data = weather.json()

        return jsonify({
            "location": {
                "name": place.get("name"),
                "country": place.get("country"),
                "latitude": lat,
                "longitude": lon
            },
            "current": data.get("current", {}),
            "daily": data.get("daily", {})
        })

    except requests.RequestException:
        return jsonify({"error": "Weather service is temporarily unavailable."}), 503
    except (KeyError, TypeError, ValueError):
        return jsonify({"error": "Unexpected weather data received."}), 502

if __name__ == "__main__":
    app.run(debug=True)
```

4. Frontend Source Code – index.html (Part 1)

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI WeatherWise</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <main class="container">
        <h1>AI WeatherWise</h1>
        <p class="subtitle">Simple Weather API Dashboard</p>

        <div class="search-box">
            <input id="city" type="text" placeholder="Enter city name">
            <button onclick="getWeather()">Search</button>
        </div>

        <p id="message"></p>

        <section id="result" class="hidden">
            <div class="card">
                <h2 id="location"></h2>
                <div class="main-temp" id="temperature"></div>
                <p id="details"></p>
            </div>

            <h2>5-Day Forecast</h2>
            <div id="forecast" class="forecast"></div>
        </section>
    </main>

    <script>
        function weatherText(code) {
            const map = {
                0: "Clear sky",
                1: "Mainly clear",
                2: "Partly cloudy",
                3: "Overcast",
                45: "Fog",
                48: "Rime fog",
                51: "Light drizzle",
                61: "Rain",
                63: "Moderate rain",
                65: "Heavy rain",
                71: "Snow",
                80: "Rain showers",
                95: "Thunderstorm"
            };
            return map[code] || "Weather condition";
        }

        async function getWeather() {
            const city = document.getElementById("city").value.trim();
            const message = document.getElementById("message");
            const result = document.getElementById("result");

            result.classList.add("hidden");
```

5. Frontend Source Code – index.html (Part 2)

```
            if (!city) {
                message.textContent = "Please enter a city name.";
                return;
            }

            message.textContent = "Loading...";

            try {
                const response = await fetch(
                    `/weather?city=${encodeURIComponent(city)}`
                );
                const data = await response.json();

                if (!response.ok) {
                    throw new Error(data.error || "Request failed");
                }

                document.getElementById("location").textContent =
                    `${data.location.name}, ${data.location.country}`;

                document.getElementById("temperature").textContent =
                    `${data.current.temperature_2m} °C`;

                document.getElementById("details").textContent =
                    `${weatherText(data.current.weather_code)} | ` +
                    `Humidity: ${data.current.relative_humidity_2m}% | ` +
                    `Wind: ${data.current.wind_speed_10m} km/h`;

                const daily = data.daily;
                const forecast = document.getElementById("forecast");
                forecast.innerHTML = "";

                daily.time.forEach((date, i) => {
                    const card = document.createElement("div");
                    card.className = "forecast-card";
                    card.innerHTML = `
                        <strong>${date}</strong>
                        <p>${weatherText(daily.weather_code[i])}</p>
                        <p>Max: ${daily.temperature_2m_max[i]} °C</p>
                        <p>Min: ${daily.temperature_2m_min[i]} °C</p>
                        <p>Rain chance: ${daily.precipitation_probability_max[i]}%</p>
                    `;
                    forecast.appendChild(card);
                });

                message.textContent = "";
                result.classList.remove("hidden");
            } catch (error) {
                message.textContent = error.message;
            }
        }

        document.getElementById("city").addEventListener("keydown", (e) => {
            if (e.key === "Enter") getWeather();
        });
    </script>
</body>
</html>
```

6. CSS Source Code – static/style.css

```
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #eef4ff;
    color: #172033;
}

.container {
    width: min(1000px, 92%);
    margin: 40px auto;
}

h1 {
    text-align: center;
    margin-bottom: 5px;
}

.subtitle {
    text-align: center;
    color: #56627a;
}

.search-box {
    display: flex;
    gap: 10px;
    margin: 30px 0;
}

input {
    flex: 1;
    padding: 14px;
    border: 1px solid #b8c4d9;
    border-radius: 8px;
    font-size: 16px;
}

button {
    padding: 14px 22px;
    border: 0;
    border-radius: 8px;
    cursor: pointer;
    background: #2563eb;
    color: white;
    font-weight: bold;
}

.card, .forecast-card {
    background: white;
    border-radius: 12px;
    padding: 20px;
    box-shadow: 0 4px 16px rgba(0,0,0,0.08);
}

.main-temp {
    font-size: 48px;
    font-weight: bold;
    margin: 15px 0;
}

.forecast {
    display: grid;
    grid-template-columns: repeat(5, 1fr);
    gap: 12px;
}

.forecast-card p {
    margin: 8px 0;
}

#message {
    min-height: 24px;
    text-align: center;
}

.hidden {
    display: none;
}

@media (max-width: 750px) {
    .search-box {
        flex-direction: column;
    }

    .forecast {
        grid-template-columns: 1fr;
    }
}
```

7. Dependencies and Run Scripts

requirements.txt

```
Flask>=3.0
requests>=2.31
```

Windows – run_windows.bat

```
@echo off
python -m venv venv
call venv\Scripts\activate
pip install -r requirements.txt
python app.py
pause
```

Linux/macOS – run_linux_mac.sh

```
#!/bin/bash
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python app.py
```

8. Easy Installation and Execution

Install Python 3.10+. Open a terminal in the project folder.

```
python -m venv venv
Windows: venv\Scripts\activate
Linux/macOS: source venv/bin/activate
pip install -r requirements.txt
python app.py
```

Open **http://127.0.0.1:5000** in a browser. Enter a city and press Search. Test with Coimbatore, Chennai, Bengaluru or another valid city.

9. API Flow and Weather Variables

1. User enters a city. 2. Browser calls /weather. 3. Flask queries geocoding. 4. Coordinates are obtained. 5. Flask queries forecast. 6. JSON is validated. 7. Selected values are returned. 8. JavaScript renders the dashboard.

Current variables

```
temperature_2m
relative_humidity_2m
apparent_temperature
precipitation
weather_code
wind_speed_10m
```

Daily variables

```
weather_code
temperature_2m_max
temperature_2m_min
precipitation_probability_max
```

10. Testing and Troubleshooting

Test cases

```
TC01 Valid city       → Weather displayed
TC02 Empty input      → Validation message
TC03 Unknown city     → Location not found
TC04 Internet failure → Service unavailable
TC05 Bad API data     → Safe error response
TC06 Mobile screen    → Responsive layout
```

ModuleNotFoundError: activate venv and install requirements. **Port busy:** stop another Flask process/change port. **Location not found:** use a standard city name. **Service unavailable:** check internet. **CSS missing:** confirm static/style.css.

11. Project Conclusion

The prototype demonstrates web development, REST API integration, JSON processing and responsive weather visualization. It is intentionally simple so a beginner can install the dependencies and start the dashboard with one command.

Future enhancements can include charts, database storage, historical analysis, machine-learning classification, multilingual support, alerts and mobile/PWA deployment. These should be separately evaluated and clearly distinguished from upstream weather forecasts.

12. References & Quick Checklist

References

1. Open-Meteo Forecast API – https://open-meteo.com/en/docs
2. Open-Meteo Geocoding API – https://open-meteo.com/en/docs/geocoding-api
3. Python – https://docs.python.org/
4. Flask – https://flask.palletsprojects.com/
5. Requests – https://requests.readthedocs.io/

Checklist

```
[ ] Python installed
[ ] Folders created
[ ] app.py copied
[ ] index.html copied
[ ] style.css copied
[ ] requirements.txt copied
[ ] pip install -r requirements.txt
[ ] python app.py
[ ] Open http://127.0.0.1:5000
[ ] Test a city and capture screenshots
```

Ready for local execution: this implementation does not require a weather API key.